

ARTI403 – Lab 2

Digital Image Fundamentals

Name: Ali Jafar Altaweel

ID: 2240006456

Section: 7MA2

GitHub: <https://github.com/Ali-developer/Image-Processing/tree/main/Lab2>

In this lab, I worked on the two operations that turn a continuous image into a digital one, sampling and quantization, and then on the arithmetic, logical and set operations that can be applied to images. The main objective was to understand how an image is represented as an array of sampled and quantized numbers, and how those numbers behave when they are combined.

Lab Tasks

Task 1 – Sampling and quantization

Change the sampling and quantization parameters and observe the effects.

Task 2 – Operations on two images

Read two images, convert them into arrays, and perform the following operations on them:

- Subtract two images and display the result.
- Add one image with a constant value of 175 and display it.
- Apply the set difference operation on two Gray-Scale images.
- Apply the symmetric difference operation on two Gray-Scale images.
- Apply Intersection operations on two Gray-Scale images.

Images Used

I used `lena_gray_256.tif`, a 256×256 8-bit grayscale image, and `cameraman.tif`, a 512×512 8-bit grayscale image. These are the two classic test images of digital image processing.

For the set operations the manual also uses two images called `A.png` and `B.png`, which are not shipped with it. I created them as two overlapping binary shapes, a filled square and a filled disc holding only the values 0 and 255, which is the usual way of illustrating union, intersection and difference.

I made both of them 400×400 , which is exactly the size the manual resizes them to. This matters, because in a first attempt they were 256×256 and the LANCZOS up-scaling introduced intermediate gray values and ringing at the edges. After that the images were no longer binary and the logical operators no longer produced clean shapes.

Sampling and Quantization

I defined the three functions given in the manual. `sample_image()` uses `cv2.resize()` with nearest-neighbor interpolation to keep one pixel out of every `*factor*` pixels, and `quantize_image()` rounds every gray value down onto a coarser grid using `np.floor()`. The third function plots the original, sampled and quantized images side by side.

Using the parameters from the manual, a sampling factor of 14 and 9 quantization levels, I obtained the following results.

Sampling changed only the shape of the image. The 256×256 image, which holds 65 536 pixels, became an 18×18 image holding 324 pixels, which is about 202 times less data. Every surviving pixel keeps its exact original value, so the gray levels are untouched, but the spatial detail is gone.

Quantization changed only the amplitude. The image kept all of its 65 536 pixels, but with a step of 28 between levels only 9 distinct gray values were left out of the original 210. The smooth areas of the image broke into flat patches.

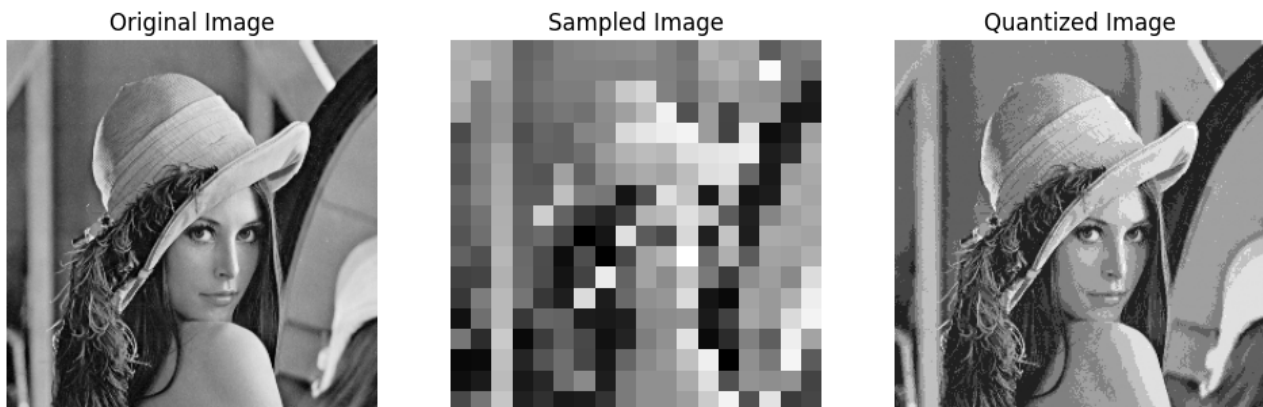


Figure 1 – Original, sampled (factor 14) and quantized (9 levels) image

Arithmetic Operations

I opened Lena and the cameraman with Pillow, resized both to 400×400 so that the arrays had the same shape, converted them to arrays and added them together.

The result was not what I expected at first. Adding the two arrays directly gave an image covered with dark speckles exactly where both images were bright. The reason is that the arrays are of type `uint8`, so the addition is computed modulo 256 and any value above 255 wraps back around to 0. In this case 86 212 pixels out of 160 000 exceeded 255, so more than half of the image was affected.

I compared three ways of adding the images:

- Direct addition, which wraps around and turns bright pixels black
- `cv2.add()`, which saturates everything above 255 to 255
- Averaging the two images, which keeps the result inside the valid range by construction



Figure 2 – Adding the two images directly (uint8 wrap-around)



Figure 3 – Three ways of adding the images

Set and Logical Operations

I opened the two binary images and combined them with the $|$ operator, which is the union of the two sets of white pixels. The result contained only the values 0 and 255, so the union of two binary images is again a binary image.

The counts confirmed the set behavior: the square had 46 656 white pixels and the disc had 36 608, but their union had 67 348 rather than 83 264, because the 15 916 pixels of the overlap are counted only once.

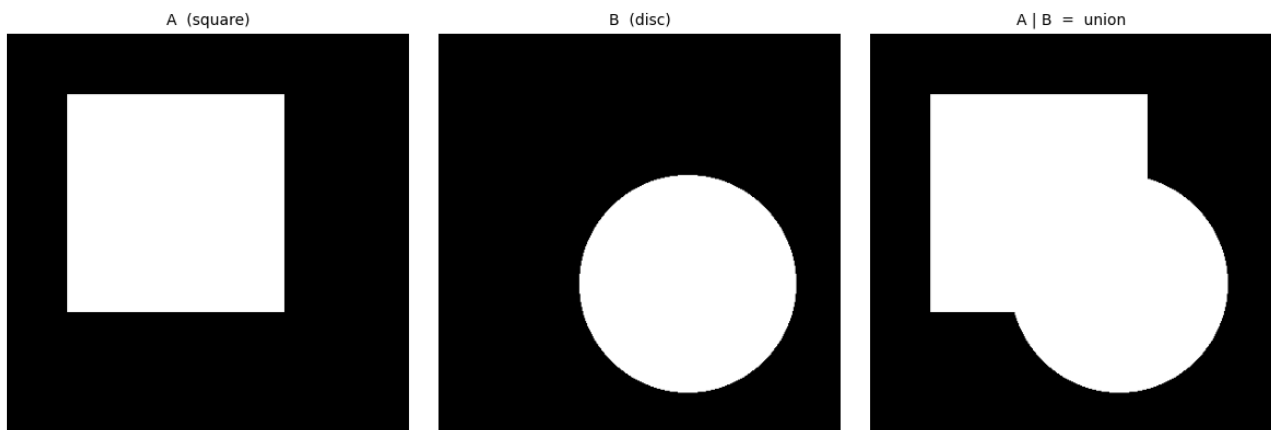


Figure 4 – The binary images A and B and their union

Changing the Sampling and Quantization Parameters

For the first task I repeated the sampling with the factors 1, 2, 4, 8, 16, 32 and 64, and the quantization with 256, 64, 16, 9, 8, 4 and 2 levels.

The amount of data falls as one over the factor squared, so a factor of 4 already stores 16 times less data. Visually the image becomes more and more blocky and its edges turn into staircases. The face is still recognizable up to a factor of 8, but at a factor of 16 the image is only 16×16 pixels and the content is lost.

Reducing the quantization levels never changes the size of the image, only the number of distinct gray values. At 64 levels the result is almost indistinguishable from the original. From 16 levels downwards the smooth regions break into visible flat bands, an effect called false contouring, and at 2 levels the image becomes pure black and white.

I also noticed that the given `quantize_image()` function does not always return the number of levels that was asked for. Because `256 // levels` is an integer division, asking for 9 levels gives a step of 28, which actually spans 10 levels. The function is exact only when the number of levels is a power of two.

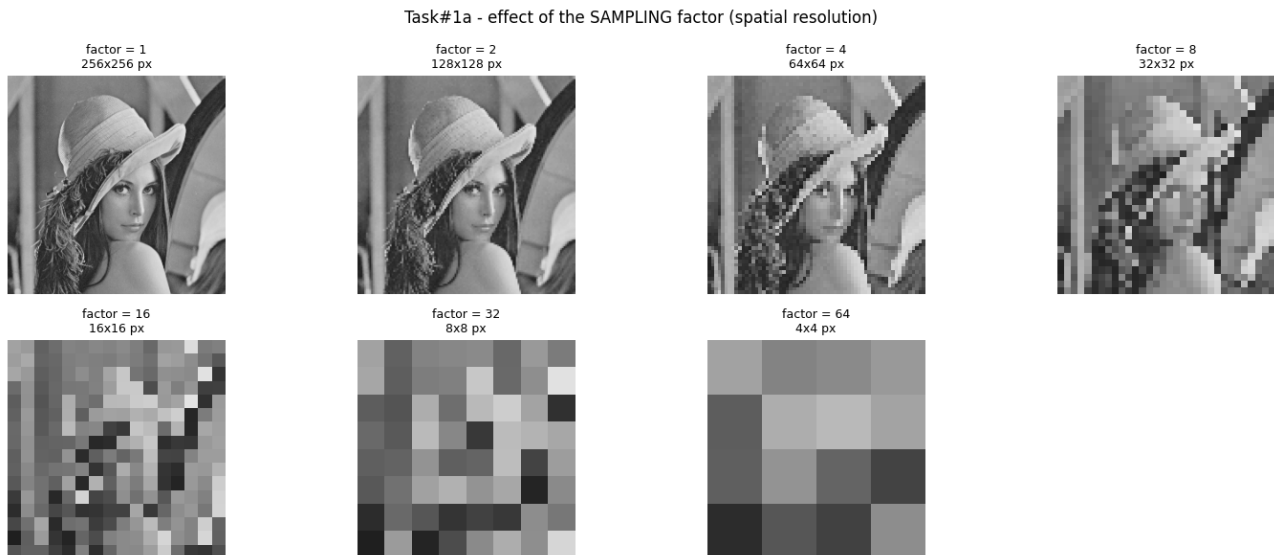


Figure 5 – Task 1: effect of the sampling factor

Task#1b - effect of the QUANTIZATION levels (gray-level resolution)



Figure 6 – Task 1: effect of the number of quantization levels

Task#1c - sampling (rows) x quantization (columns)



Figure 7 – Task 1: sampling and quantization combined

Operations on Two Images

For the second task I used the two 400×400 arrays and applied five operations. For grayscale images the set operations are defined on the gray values themselves, so the union is the maximum of the two images, the intersection is the minimum, and the complement is 255 minus the image.

Subtraction. Subtracting the two images shows where they differ, since identical pixels become 0. Here the `uint8` type caused a problem again: where the true difference is negative, the result wraps around, so a difference of -1 is stored as 255 and the darkest difference is displayed as the brightest pixel. Almost half of the pixels were affected. Using `cv2.subtract()`, which saturates to 0, or `cv2.absdiff()`, which takes the absolute value, gave a meaningful result.

Adding a constant of 175. This is a brightness increase, and the histogram of the image shifted 175 positions to the right. With saturation the 125 243 pixels that went above 255 all piled up in the last

bin, so the image became almost white and the detail in the bright areas was lost. With the wrap-around the mean actually dropped instead of rising, because the bright pixels turned black.

Set difference. Computed as the minimum of the image and the complement of the other one, this keeps the gray value of the first image only where the second one is dark. I verified numerically that the result changes when the two images are swapped, which confirms that the set difference is not commutative.

Symmetric difference. Computed as the maximum of the two differences, this is bright where the two images disagree and dark where they agree. Unlike the plain difference, it is commutative.

Intersection. Computed as the minimum of the two images, this keeps the darker of the two values at every position, so it is bright only where both images are bright. Its mean was below both input means, while the mean of the union was above both. I also checked that the minimum plus the maximum equals the sum of the two images, which held on all 160 000 pixels.

I then repeated the same five operations on the binary square and disc, where the shapes make each result immediately readable, and confirmed that the minimum and maximum definitions give exactly the same arrays as the bitwise operators $|$, $\&$, $\wedge$ and $\sim$.

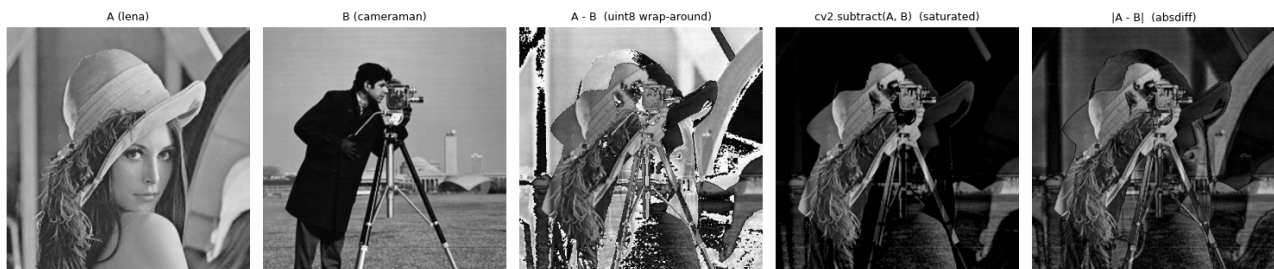


Figure 8 – Task 2: subtraction of the two images



Figure 9 – Task 2: adding the constant 175



Figure 10 – Task 2: set difference



Figure 11 – Task 2: symmetric difference

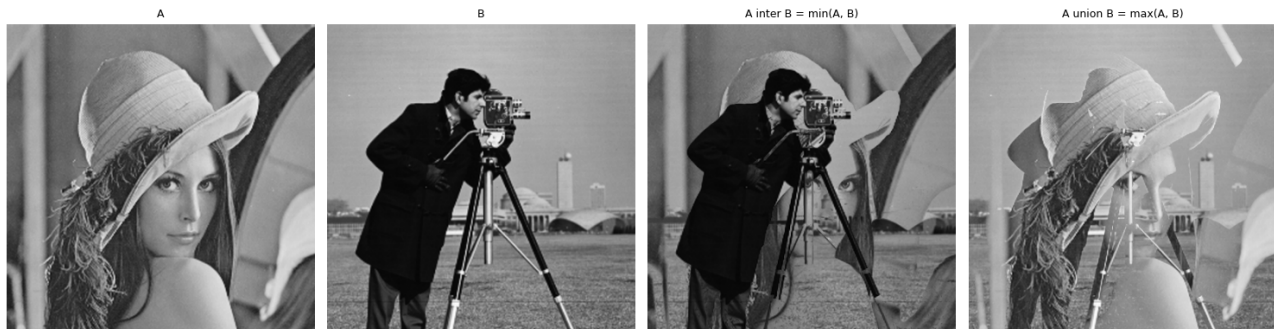


Figure 12 – Task 2: intersection and union

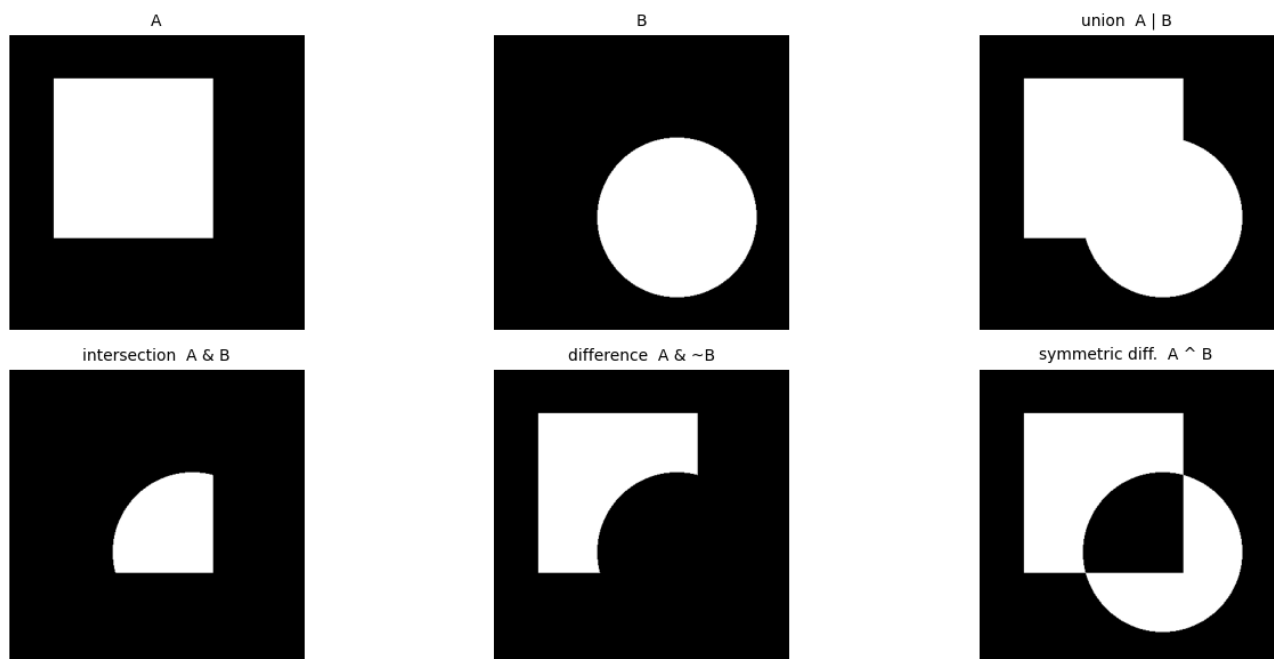


Figure 13 – Task 2: the same operations on the binary images

Observations

The most important observation of this lab is that the `uint8` data type has to be watched constantly. Three separate operations, adding two images, subtracting two images, and adding a constant, all gave visually wrong results because NumPy computes them modulo 256. In every case the fix was either to use the OpenCV function that saturates instead of wrapping, or to convert to a wider type before clipping back.

The second observation is that sampling and quantization degrade the image in completely different and independent ways. Sampling attacks the coordinates and produces blockiness, quantization attacks the amplitude and produces false contours. Combining a moderate amount of both is what

makes an image cheap to store while still recognizable.

Conclusion

This lab demonstrated how sampling and quantization convert an image into a digital one, and how the choice of those two parameters trades image quality against storage cost. Reducing the sampling factor by 4 and the levels to 4 shrank the test image from 65 536 bytes to about 1 024 bytes, at the cost of most of the detail.

It also showed that once an image is an array, every arithmetic, logical and set operation is a one-line NumPy expression, provided the data type is handled carefully. The grayscale definitions of the set operations, based on the minimum and the maximum, were verified to be the correct generalization of the ordinary logical operators used on binary images.

Submission

The complete work for this lab is in my GitHub repository, in the **Lab2** folder: <https://github.com/Ali-develober/Image-Processing/tree/main/Lab2>

As required, the Lab2 folder contains the following files:

- Jupyter notebook: Lab2.ipynb, executed, with all the outputs and the task solutions
- Images used: images/A.png, images/B.png, images/cameraman.tif, images/lena_gray_256.tif
- This report: Lab2_Report.pdf, and the same write-up as README.md